> *let's move on to step by step implementation of the base game with minimal sprite usage. Divide the implementation into several phases with completing the base game as part of the core phase. Following phases will focus on polishing the game to make it fit for market deploy.*

To build **The Wenderer** from your starter triangle code into a complete, deployable game, let's break the development down into four structured phases. I will help guide you through each step with questions and code milestones along the way.

🛠️ Development Roadmap

- **Phase 1: Core Engine & Base Game (Minimal Sprites)** 🧱
  We will set up the 2D/pseudo-3D coordinate space using `GLM`, create a single reusable quad (two triangles) for all objects, and implement player movement, obstacle spawning, Axis-Aligned Bounding Box (AABB) collisions, and the core "Wending" (90-degree directional shift and camera rotation) mechanic. Everything will use solid colors or basic geometric shapes first.

- **Phase 2: Visuals, Sprites & Audio Integration** 🎨
  We will replace solid-colored quads with texture atlases using `stb_image`, add UV offset animations in the vertex shader, and hook up event-driven sound effects (SFX) for turns and impacts.

- **Phase 3: Philosophical Polish & "Game Feel"** 🌌
  To embody the theme of human adaptability and searching for meaning, we will add atmospheric lighting/color transitions as the player endures more redirections, smooth camera interpolation (`glm::mix`), particle bursts, and screen shake.

- **Phase 4: Market Readiness & Deployment** 🚀
  We will implement game states (Title Screen, Pause, Game Over), score tracking, frame-rate independent optimization, and package the minimal C++ binary (`< 10 MB) for distribution.

Starting Phase 1: Moving from Triangles to a Reusable Quad

In your `main.cpp` file, you currently define three separate triangles using three different VAOs and VBOs (`VAOs[3]`, `VBOs[3]`), and their positions are hardcoded on the screen.

For our base game, the player, track segments, and obstacles can all be drawn as rectangles (quads). Instead of creating a new VAO for every single obstacle that spawns, we only need **one VAO** representing a standard 1×1 square centered at $(0, 0)$. We can then move, resize, and rotate that single square anywhere on screen using a **Model Matrix** (M) inside our vertex shader.

Looking at how `firstTriangle` and `secondTriangle` are set up in your `main.cpp`:

```
float firstTriangle[] = {
    -0.5f, 0.5f,  0.0f, // top-left
    0.5f,  0.5f,  0.0f, // top-right
    0.5f,  -0.5f, 0.0f, // bottom-right
};
float secondTriangle[] = {
    -0.5f, 0.5f,  0.0f, // top-left
    -0.5f, -0.5f, 0.0f, // bottom-left
    0.5f,  -0.5f, 0.0f  // bottom-right
};
```

If we combine these two triangles into a single `quadVertices[]` array inside one VBO, how many total vertices (and `float` values) will we pass into `glDrawArrays`, and what change do we need to make to `vertexShaderSource` so that we can move this quad around dynamically each frame?

> *` give me the complete steps for implementing "Phase 1: Core Engine & Base Game (Minimal Sprites)"*

To complete **Phase 1: Core Engine & Base Game (Minimal Sprites)**, we will transform your single-file `main.cpp` into a working endless runner prototype using five concrete milestones:

1. **Step 1.1: The Reusable Quad & Shader Setup** 🟦
   Combine `firstTriangle` and `secondTriangle` into a single 1×1 quad VAO, and replace your three separate fragment shaders with a single shader that accepts a `uniform vec3 uColor` and a `uniform mat4 uMVP` matrix.

2. **Step 1.2: The Game Loop & Delta Time (Δt)** ⏱️
   Add frame timing (`glfwGetTime()`) inside your `while (!glfwWindowShouldClose(window))`

loop so the player moves at a consistent speed on any computer.

3. **Step 1.3: Player Movement & Vector-Adaptive Controls** 🏃
   Track the player's position (`glm::vec2`), forward running direction, and lane-switching/jumping inputs relative to the current axis.

4. **Step 1.4: Obstacle Spawning & AABB Collision** 🚧
   Maintain a `std::vector` of obstacle rectangles ahead of the player and check for Axis-Aligned Bounding Box (AABB) overlaps every frame.

5. **Step 1.5: The "Wending" Mechanic & Camera Rotation** 🔄
   Trigger a 90-degree directional turn at set distance intervals and rotate the View matrix (`glm::rotate`) to swing the camera onto the new path.

Let's Build Step 1.1: The Reusable Quad & Shader

In your `main.cpp` , you created three separate shader programs (`shaderProgramOrange` , `shaderProgramYellow` , `shaderProgramRed`) just to draw three colors. In a game, we use **one shader program** and pass both the object's transformation and color from C++ using `uniform` variables.

Here is the updated Vertex and Fragment shader code to replace `vertexShaderSource` and `fragmentShader1Source` in your `main.cpp` :

```
const char *vertexShaderSource =
    "#version 330 core\n"
    "layout (location = 0) in vec3 aPos;\n"
    "uniform mat4 uMVP;\n" // Combines Projection * View * Model
    "void main()\n"
    "{\n"
    "   gl_Position = uMVP * vec4(aPos, 1.0);\n"
    "}\0";

const char *fragmentShaderSource =
    "#version 330 core\n"
    "out vec4 FragColor;\n"
    "uniform vec3 uColor;\n" // Dynamically set color from C++
    "void main()\n"
    "{\n"
```

```
    "   FragColor = vec4(uColor, 1.0f);\n"
    "}\n\0";
```

And instead of three VBOs/VAOs, we combine `firstTriangle` and `secondTriangle` into one `quadVertices` array of **6 vertices** (18 floats):

```
float quadVertices[] = {
    // First triangle
    -0.5f,  0.5f, 0.0f, // Top-Left
     0.5f,  0.5f, 0.0f, // Top-Right
     0.5f, -0.5f, 0.0f, // Bottom-Right
    // Second triangle
    -0.5f,  0.5f, 0.0f, // Top-Left
    -0.5f, -0.5f, 0.0f, // Bottom-Left
     0.5f, -0.5f, 0.0f  // Bottom-Right
};

unsigned int quadVAO, quadVBO;
glGenVertexArrays(1, &quadVAO);
glGenBuffers(1, &quadVBO);

glBindVertexArray(quadVAO);
glBindBuffer(GL_ARRAY_BUFFER, quadVBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(quadVertices), quadVertices, GL_STATIC_DRAW);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
```

When we draw this quad inside the `while` render loop using `glDrawArrays(GL_TRIANGLES, 0, ...)`, what number should replace the `3` that you originally had in your `main.cpp`, and have you already included the **GLM** header files (`#include <glm/glm.hpp>`) in your project folder?